

Refactoring Report: Edge Detection Notebook

AI Assistant

December 9, 2025

1 Introduction

We worked with Google's Antigravity (Gemini 3 Pro) to edit HW4 Question 6 with the goal of improving code quality in terms of best practices in software engineering. While the notebook already implemented some good practices such as function docstrings, type hints, and helper functions, the first thing we noticed was how dense the notebook was. The original notebook contained all class definitions, helper functions, and training logic within the cells. This monolithic structure makes it difficult to maintain, debug, and reuse code. To address this, we have modularized the codebase by extracting logical components into separate Python files within a `src/` directory. This report details the changes made and the reasoning behind them.

2 Methodology

We gave Antigravity the prompt "i need to edit this notebook to follow better "swe" practices. this can be helping code cleanliness, modularization, etc. can you help me with this task?" Despite the vague instructions, it was able to complete the task in one shot. Along with the edits, it gave a detailed explanation of its changes, as summarized in the following section.

3 Changes

The following file structure was created:

- `src/dataset.py`: Contains the `EdgeDetectionDataset` class.
- `src/models.py`: Contains all neural network architectures (`WiderCNN`, `DeeperCNN`, etc.).
- `src/utils.py`: Contains utility functions for training, evaluation, and logging.
- `src/visualization.py`: Contains functions for plotting training curves and visualizing datasets.
- `edge_detection.ipynb`: The main notebook, now significantly cleaner and focused on experimentation.

3.1 Dataset Modularization (`src/dataset.py`)

Passage Extracted: The `EdgeDetectionDataset` class.

Reasoning: The dataset class is a fundamental component that defines how data is generated and accessed. Moving it to a separate file allows it to be imported into other scripts or notebooks without duplicating the code. It also separates data engineering logic from model training logic.

```
1 class EdgeDetectionDataset(Dataset):
2     def __init__(self, domain_config, mode="train", transform=None) -> None:
3         # ... logic for dataset initialization ...
4         self.data_per_class = domain_config.get("data_per_class", 1000)
5         # ...
```

Listing 1: Snippet from `src/dataset.py`

3.2 Model Architectures (src/models.py)

Passage Extracted: WiderCNN, DeeperCNN, SimpleCNN, ThreeLayerCNN, TwoLayerMLP, etc.

Reasoning: Defining multiple model architectures in a single notebook cell creates clutter. By moving them to `models.py`, we create a centralized model registry. This makes it easier to compare different architectures and ensures that model definitions remain consistent across experiments.

```
1 class SimpleCNN(nn.Module):
2     def __init__(self, num_filters=3, kernel_size=2, num_classes=3):
3         super().__init__()
4         # ... layer definitions ...
5
6     def forward(self, x):
7         # ... forward pass ...
8         return x
```

Listing 2: Snippet from `src/models.py`

3.3 Utilities and Training Loop (src/utils.py)

Passage Extracted: `train_one_epoch`, `evaluate`, `train_model`, `set_seed`, etc.

Reasoning: The training loop is often reusable boilerplate code. Abstracting it into functions like `train_model` allows the notebook to focus on *what* is being trained (hyperparameters, data) rather than *how* it is being trained.

```
1 def train_model(model, optimizer, num_epochs, train_loader, valid_loader):
2     criterion=nn.CrossEntropyLoss()
3     # ... training loop logic ...
4     return {"final_valid_acc": valid_acc_list[-1], ...}
```

Listing 3: Snippet from `src/utils.py`

3.4 Visualization (src/visualization.py)

Passage Extracted: `vis_training_curve`, `vis_dataset`, `vis_kernel`, etc.

Reasoning: Plotting code is verbose and can distract from the narrative of the notebook. Moving these functions to a dedicated module keeps the notebook clean and ensures consistent styling for all plots.

3.5 Notebook Cleanup (edge_detection.ipynb)

The notebook was updated to import these modules. We used the `autoreload` extension to ensure that changes in the `.py` files are immediately reflected in the notebook without needing to restart the kernel.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import torch
4 # ... standard library imports ...
5
6 from src.dataset import EdgeDetectionDataset
7 from src.models import *
8 from src.utils import *
9 from src.visualization import *
10
11 %load_ext autoreload
12 %autoreload 2
```

Listing 4: Updated Notebook Imports

4 Conclusion

The refactored project structure adheres to modern software engineering standards for machine learning projects. It promotes:

- **Modularity:** Components are isolated and self-contained.

- **Reusability:** Code can be easily shared between different experiments.
- **Readability:** The notebook is now a high-level orchestration tool rather than a dumping ground for code.

Also, from a teaching/student perspective, the notebook now mainly contains the questions, context, and TODOs rather than helper function definitions. This makes the assignment experience more streamlined and focused for the student.